

Assignment 00

Charanjit Dhanasekar

August 26, 2026

Assignment 00

Due: See Date in Moodle

Guidelines

- Follow good programming practices by using descriptive variable names, maintaining appropriate spacing for readability, and adding comments to clarify your code.
- Ensure written responses use third-person, correct spelling, complete sentences, and proper grammar.

Name: Charanjit Dhanasekar

Section: 001

Date: 08/26/2026

Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see <https://quarto.org>.

Running Code

When you click the **Render** button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```
1 + 1
```

```
[1] 2
```

You can add options to executable code like this:

```
{r}
#| echo: false
2*2
```

```
[1] 4
```

The `echo: false` option disables the printing of code (only output is displayed).

You can label code chunks like this:

```
library(readxl)
library(dplyr)
library(ggplot2)
```

```
{r}
#| label: load-packages
#| message: false
#| warning: false

library(readxl)
library(dplyr)
library(ggplot2)
```

The `message: false` and `warning: false` options hide messages and warnings from the rendered output.

Loading Data

```
ms2022 <- read_excel("data/DSA 406 Annual Data 2022.xlsx")
ms2022 <- ms2022 |>
  filter(!is.na(restaurant))

glimpse(ms2022)
```

Rows: 26,237

Columns: 33

```
$ matched_2021      <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
$ new_item_2022     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ menu_item_id      <dbl> 10012, 10013, 10018, 10020, 10023, 10025, 10056~
$ food_category     <chr> "Soup", "Soup", "Soup", "Salads", "Salads", "Sa~
$ restaurant        <chr> "Applebee's", "Applebee's", "Applebee's", "Appl~
$ item_name          <chr> "French Onion Soup", "Tomato Basil Soup", "Chic~
$ item_description   <chr> "French Onion Soup, Soups", "Tomato Basil Soup,~
$ serving_size       <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ serving_size_text  <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ serving_size_unit  <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ serving_size_household <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ calories           <dbl> 370, 220, 280, 130, 1560, 230, 1430, 1450, 180,~
$ total_fat          <dbl> 22, 12, 15, 7, 103, 18, 89, 80, 9, 106, 66, 75,~
$ saturated_fat      <dbl> 12.0, 4.5, 4.0, 2.5, 17.0, 3.5, 14.0, 21.0, 2.0~
$ trans_fat          <dbl> 0.0, 1.5, 0.0, 0.0, 1.0, 0.0, 1.0, 0.5, 0.0, 2.~
$ cholesterol        <dbl> 55, 25, 35, 10, 65, 15, 150, 120, 20, 200, 85, ~
$ sodium             <chr> "1250", "1270", "930", "230", "1610", "410", "2~
$ carbohydrates      <chr> "26", "22", "26", "14", "120", "13", "104", "12~
$ dietary_fiber       <dbl> 2, 2, 2, 2, 12, 2, 11, 9, 1, 8, 7, 9, 1, 2, 7, ~
$ sugar              <dbl> 9, 9, 3, 4, 44, 3, 52, 21, 5, 6, 7, 6, 0, 10, 1~
$ protein            <chr> "16", "5", "11", "6", "40", "5", "56", "56", "7~
$ potassium          <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ notes              <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ calories_text      <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ total_fat_text     <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ saturated_fat_text <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ trans_fat_text     <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ cholesterol_text   <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ sodium_text        <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ carbohydrates_text <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ dietary_fiber_text <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ sugar_text         <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ protein_text       <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
```

[MenuStat](#) is a free nutrition database that aggregates menu and nutrition information from major U.S. chain restaurants, allowing users to compare food items across restaurants and analyze changes in nutritional content over time.

To learn more about the dataset and its variables, review the [Menu Stat Datasheet](#).

```
head(ms2022)
```

```
# A tibble: 6 x 33
  matched_2021 new_item_2022 menu_item_id food_category restaurant item_name
      <dbl>         <dbl>      <dbl> <chr>          <chr>      <chr>
1           1             0      10012 Soup        Applebee's French Onion~
2           1             0      10013 Soup        Applebee's Tomato Basil~
3           1             0      10018 Soup        Applebee's Chicken Tort~
4           1             0      10020 Salads      Applebee's House Salad
5           1             0      10023 Salads      Applebee's Oriental Chi~
6           1             0      10025 Salads      Applebee's Caesar Salad
# i 27 more variables: item_description <chr>, serving_size <chr>,
#   serving_size_text <lgl>, serving_size_unit <chr>,
#   serving_size_household <chr>, calories <dbl>, total_fat <dbl>,
#   saturated_fat <dbl>, trans_fat <dbl>, cholesterol <dbl>, sodium <chr>,
#   carbohydrates <chr>, dietary_fiber <dbl>, sugar <dbl>, protein <chr>,
#   potassium <dbl>, notes <lgl>, calories_text <lgl>, total_fat_text <lgl>,
#   saturated_fat_text <lgl>, trans_fat_text <lgl>, cholesterol_text <lgl>, ...
```

Question 1. Find the 10 restaurants with the fewest menu items in the dataset.

```
# Count how many menu items each restaurant has, sort from most
# to fewest, then keep the last 10 rows (the 10 restaurants with
# the fewest menu items).
restaurants <- ms2022 |>
  count(restaurant) |>
  arrange(desc(n)) |>
  tail(10)

restaurants
```

```
# A tibble: 10 x 2
  restaurant      n
    <chr>      <int>
1 Carls Jr      105
2 Sbarro        104
3 The Capital Grille 100
4 Wingstop      96
5 In-N-Out      89
6 Little Caesar's 85
7 Popeyes       79
```

8 El Pollo Loco	77
9 Five Guys	36
10 Raising Cane's Chicken Fingers	25

Question 2. Compute summary statistics for the `calories` variable.

```
# Basic summary statistics (min, quartiles, mean, max) for calories
summary(ms2022$calories)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NAs
0.0	110.0	240.0	387.9	490.0	13734.0	456

Question 3. Create a histogram showing the distribution of calories for all menu items across restaurants. Mark the maximum calorie value on the histogram.

```
# Find the highest calorie value, and create histogram
max_calories <- max(ms2022$calories, na.rm = TRUE)

ggplot(ms2022, aes(x = calories)) +
  geom_histogram() +
  geom_point(aes(x = max_calories, y = 0), color = "red") +
  labs(x = "Calories", y = "Frequency") +
  theme_classic()
```

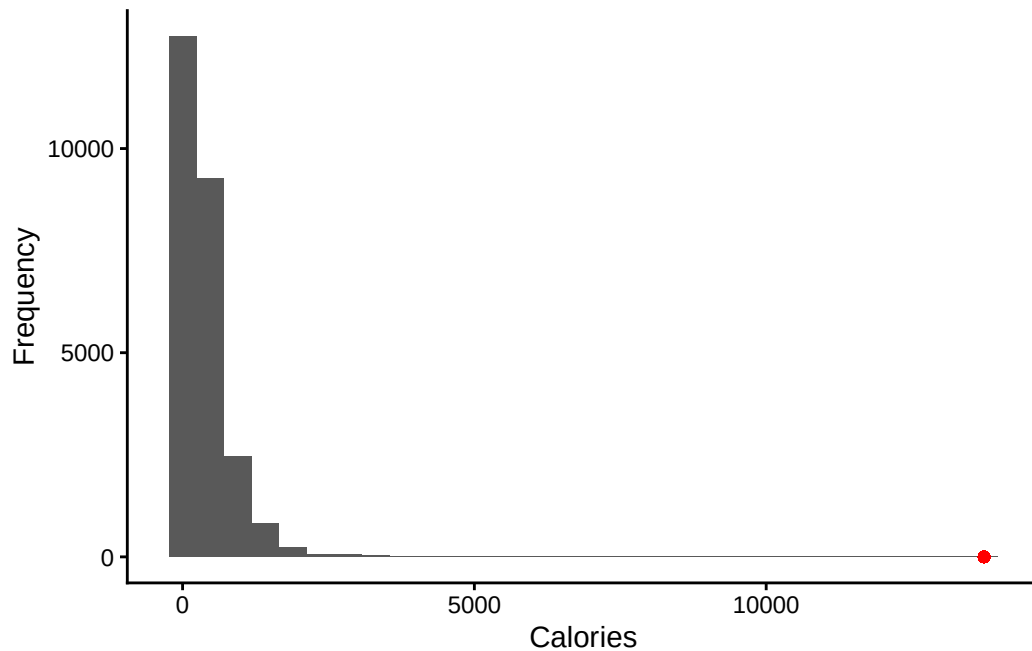


Figure 1: Calorie distribution of all menu items across restaurants.

Question 4. Find the menu item with the highest number of calories.

```
ms2022 |>
  filter(calories == max(calories, na.rm = TRUE)) |>
  select(restaurant, item_name, item_description, calories)
```

```
# A tibble: 1 x 4
  restaurant      item_name      item_description      calories
  <chr>          <chr>          <chr>          <dbl>
1 Dickey's Barbeque Party Pack, 12 Servings Party Pack (12 Servings) w~ 13734
```

Question 5. Find the highest-calorie menu item for each restaurant identified in **Question 1**.

```
highest_cal_restaurant <- ms2022 |>
  filter(restaurant %in% restaurants$restaurant) |>
  group_by(restaurant) |>
  slice_max(calories, n = 1) |>
  select(restaurant, item_name, calories) |>
  arrange(desc(calories))
```

```
highest_cal_restaurant
```

```
# A tibble: 10 x 3
```

```
# Groups:   restaurant [10]
```

	restaurant	item_name	calories
	<chr>	<chr>	<dbl>
1	Sbarro	Stuffed Sausage and Pepperoni Pizza	7200
2	Little Caesar's	5 Meat Feast, Detroit-Style Deep Dis~	3500
3	The Capital Grille	Roasted Chicken Breast w/Parmesan Ri~	2620
4	Raising Cane's Chicken Fingers	Caniac Combo	1790
5	Carls Jr	Triple Cheeseburger & Spicy Chicken ~	1430
6	Five Guys	Fries, Large	1310
7	Popeyes	Chicken Livers	1190
8	Wingstop	Louisiana Voodoo Fries Large	1180
9	El Pollo Loco	Chips & Queso (regular)	1080
10	In-N-Out	Double-double w/ Onion	670

Question 6. Create a bar chart showing the highest-calorie menu item for each restaurant identified in **Question 1**.

```
ggplot(highest_cal_restaurant, aes(x = reorder(restaurant, calories), y = calories)) +  
  geom_col() +  
  coord_flip() +  
  labs(x = "", y = "Calories") +  
  theme_classic()
```

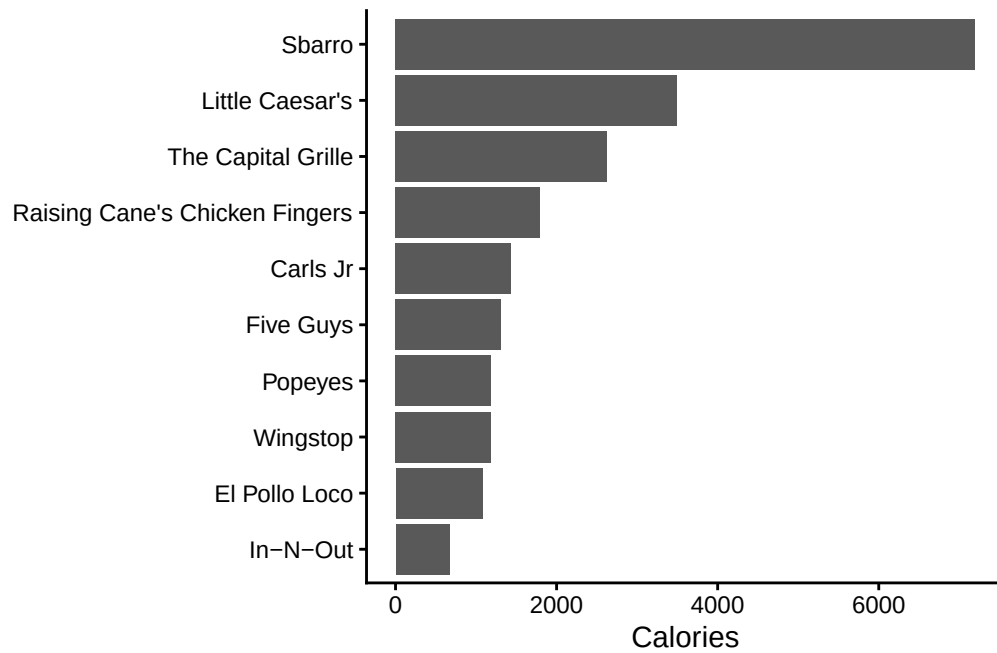


Figure 2: Highest-calorie menu item for selected restaurants.

Question 7. Create a boxplot showing the calorie distribution for menu items from the top five restaurants identified in **Question 1**.

```
ms2022 |>
  filter(restaurant %in% restaurants$restaurant[1:5]) |>
  ggplot(aes(x = restaurant, y = calories)) +
  geom_boxplot() +
  labs(x = "", y = "Calories") +
  theme_classic()
```

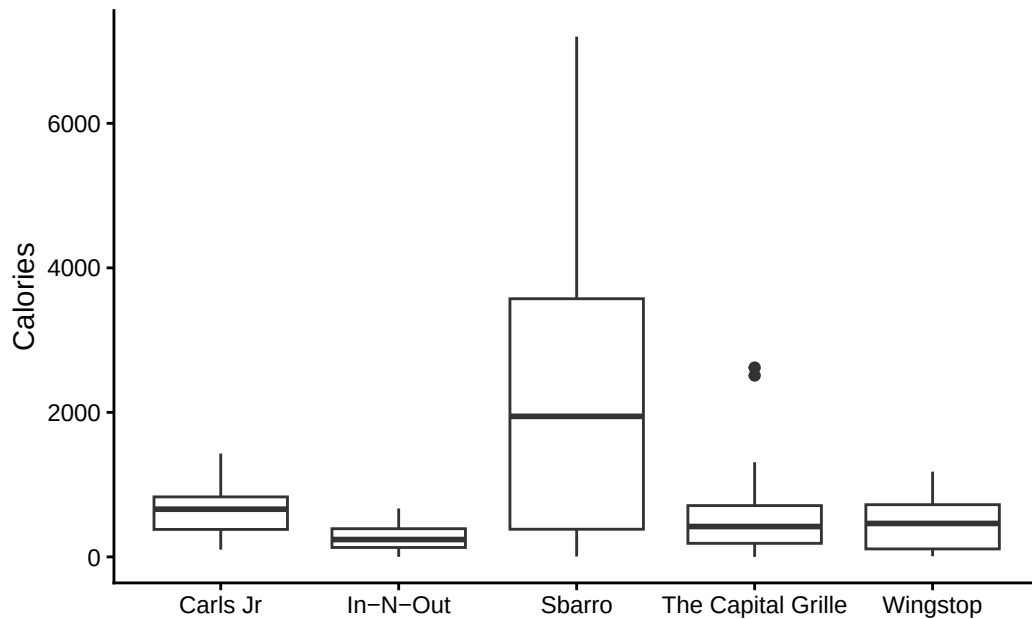



Figure 3: Calorie distribution of menu items for selected restaurants.

Question 8. Identify and inspect invalid or non-numeric values in the `sodium` variable.

```
err_sodium <- ms2022 |>
  filter(!grepl("^-?\\d*\\.?\\d+$", sodium))

unique(err_sodium$sodium)
```

```
[1] "160+"      "0+"      NA        "25-50"    "3410m"
[6] "5650m"    "-"      "0-35"    "1760\r\n\r\n" "2, 280"
[11] "1, 240"    "1, 230"  "2, 290"  "1, 290"    "3, 200"
[16] "4, 280"    "90-140"
```

```
table(err_sodium$sodium, useNA = "ifany")
```

-	0-35	0+	1, 230	1, 240	1, 290
4	1	2	1	1	1
160+ 1760\r\n\r\n		2, 280	2, 290	25-50	3, 200
1	1	1	1	1	1

3410m	4, 280	5650m	90-140	<NA>
1	1	1	1	965

Question 9. Clean and summarize the `sodium` variable by keeping only numeric values.

```
clean_sodium <- ms2022 |>
  filter(grepl("^-?\\d*\\.?\\d+$", sodium)) |>
  pull(sodium) |>
  as.numeric()

summary(clean_sodium)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.0	50.0	260.0	748.1	930.0	49051.0

Question 10. Clean and summarize the `sodium` variable by keeping only numeric values. Then create a boxplot showing the calorie distribution for menu items from the top five restaurants identified in **Question 1**.

```
ms2022 |>
  filter(grepl("^-?\\d*\\.?\\d+$", sodium),
         restaurant %in% restaurants$restaurant[1:5]) |>
  mutate(clean_sodium_col = as.numeric(sodium)) |>
  ggplot(aes(x = restaurant, y = clean_sodium_col)) +
  geom_boxplot() +
  labs(x = "", y = "Sodium") +
  theme_classic()
```

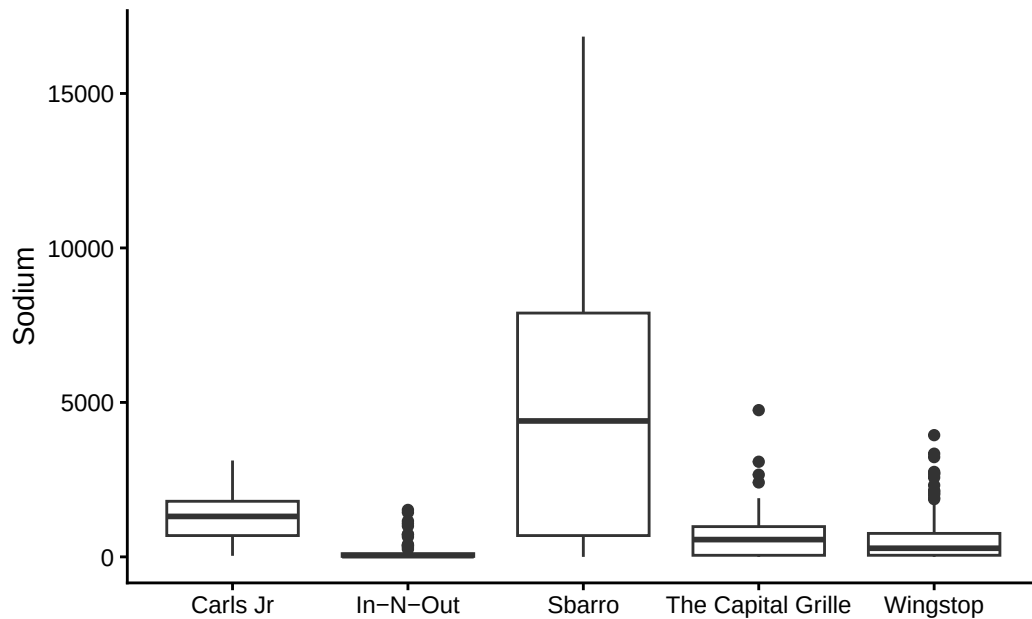


Figure 4: Sodium distribution of menu items for selected restaurants.

Submission

Make sure that all cells in your assignment have been executed to display all output, images, and graphs in the final document.

Note: Save the assignment before proceeding to render the file into a **.pdf** document.

Once the rendering process is complete, locate the resulting **.pdf** document and upload **only** this file to Moodle. The assignment will be automatically submitted to Gradescope for grading.

If you have any questions, please post them on the Moodle discussion board dedicated to this assignment.

Grading Rubric

This programming assignment is worth **15 points**: 10 points for the questions, 4 points for submission compliance, and 1 point for timeliness. See the [Grading Rubric Guideline](#) for details on marking tags and point values.